

CSC 330: Assignment 2

Deadline: Thu, Feb 5 at 23.59

Submit via: [CSC 330 Submission Portal](#)

Ground rules

- You must complete this assignment *entirely* on your own. In other words, you should come up with the solution *yourself*, write the code *yourself*, and test the code *yourself*. The university policies on academic dishonesty (a.k.a. cheating) will be taken very seriously. Check the syllabus for details on what is allowed and what is not.
 - You can have high-level discussions with your classmates about the course material.
 - You are also welcome to use Teams or come to office hours and ask questions. If in doubt, please ask—we are here to help!
 - Using LLMs to get the solutions is akin to shooting yourself in the foot: you *might* pass the assignment, but you will most likely fail the written exams.
- You can use a maximum of **three (3) late days per assignment**, and a maximum of **seven (7) days for the whole semester**. Any further accommodations must be announced and negotiated with the instructor at least seven (7) days before the deadline.
- You will need to gather at least half of the total assignment score to pass the course.
- If you think some things should be explained in detail, please include the explanations in the form of source code comments.
- We will try to grade your assignments within seven (7) days of the initial submission deadline.
- If you think there is a problem with your grade, you have one week to raise concerns after the grades go public. TAs will be holding office hours during those seven days to address any such problems. After that, your grade is set in stone.
- You should only submit a single ZIP file containing two OCaml files in .ml format to the CSC 330 Submission Portal. The contents of the ZIP must include `prob1.ml` and `prob2.ml` (lowercase).
 - Warning: different file names might result in zero points. Our scripts expect the correct file names.
 - Another warning: put the filenames at the root of the ZIP file; if you put them within a directory, our script might not be able to find it.
 - **Important:** please submit the final ZIP, as well as the submission ID obtained from the portal, via Brightspace (to ensure that there is a proper backup in case the submission portal suffers a failure).
- Document your code and include unit tests!
- Watch your style! We will penalize submissions with inadequate or bad style.
- Bonus problems can be used to improve your overall score; however, be aware that bonus problems might be harder than the rest. Bonus points cannot be transferred to other assignments or the final project.
- Your programs should generate **no** errors or warnings. If a program generates a single compilation error or warning, it will receive a zero.
- Please indicate your V-number and name within a comment at the top of each submitted file.
 - **Do not** include other private information (full name, your address, telephone number, medical exam tests etc.).

Furthermore, for this assignment:

- You cannot use mutation. Use of `ref`, `:=`, `!`, arrays, hashmaps and so on is forbidden.
- Make sure that your functions are named exactly as described. If your function is named `foo_v1` instead of `foo`, our test script will fail to find it, and you will get zero as a result.
- You cannot use the `open` statement.
- You cannot use any library function except the ones explicitly allowed for a given problem.
- You must not stray away from the features discussed in class. Don't use objects, functors etc.

Test examples, updates and error corrections will be available on the CSC 330 Submission Portal. Make sure to check it from time to time.

Problem 1: F*** Sets [25 points]

Lists are versatile data structures: they can be used to implement other structures, such as tuples (a two elements list), records (a list of tuples, where each tuple contains the name and the value of the field), and so on. This versatility is one of the reasons that languages such as LISP or OCaml place lists at the center stage.

Let's use lists to implement a data structure for polymorphic **functional** sets. A functional set can be defined as a list of (1) *ordered*, and (2) *unique* elements, where the order is maintained by some comparator function f . For the purpose of this problem, the set order is defined by a function `cmp` that takes two elements and returns:

- a zero if the elements are equal,
- a positive integer if the first element is greater than the second, and
- a negative integer if the first element is smaller than the second

(see [here](#) for more details).

Implement the following functions:

1. [2 points] `empty_set` that takes an order function and returns an empty functional set whose order is defined by the given function.
2. [1.5 points] `is_empty` that returns `true` if the given set is empty.
3. [1.5 points] `size` that returns the number of elements in a set.
4. [2.5 points] `insert` that takes a set and an element, and returns a **new** set that contains all elements from the given set and the given element.
5. [2.5 points] `remove` that takes a set and an element, and returns a **new** set that contains all elements from the given set *without* the given element. Raise an `EmptySet` exception if the element is not found.
6. [2.5 points] `union` that takes in two sets, and returns a new set that corresponds to the union of the given two sets.
7. [2.5 points] `intersect` that takes in two sets, and returns a new set that corresponds to the intersection of the given two sets.

Note: new sets should use the comparator from the provided set.

8. [2.5 points] `from_list` that takes in a comparison function and a list of elements, and returns a set that contains all elements from the list.
9. [2.5 points] `to_list` that takes in a set and returns a list of set elements ordered by the provided set's comparator.
10. [2.5 points] `map` that implements a map operation on top of the sets.
11. [2.5 points] `fold` that implements a left fold on top of the sets.

This time, we will use OCaml modules to implement this data structure. Your module will be called `FuncSet`, and must typecheck against the `FSET` signature provided below. The exact implementation of the type `set`, as well as the rest of the module, is left to you.

Bonus

12. [2 points] Are there any issues with the signature above? Is there any undefined or ambiguous behaviour that cannot be described by the signature above? Please explain what you think and put your discussion in a comment.

Hint: start by dissecting `union`.

13. [1 point] Function `map` is defined in the standard library as `('a → 'b) → 'a list → 'b list`, but in our module it is instead defined as `val map : ('a → 'a) → 'a set → 'a set`. Why the difference?
14. [2 points] Define an infix operator `++` for set building *outside the module* that will allow you to construct sets. An example:

```
empty_set Int.compare ++ 1 ++ 2 ++ 3 ++ 1 (* you get the set {1, 2, 3} *)
```

Hint: functional composition. 'nuff said.

Hints

- You need to implement *at least* `empty_set` and `to_list` to pass any test and get *any* points on this problem.
- Ensure that all functions maintain the set invariant. In plain English, make sure that no function returns a set that contains two or more identical elements, and don't forget the order.
- The expected type signatures are:

```
module type FSET = sig
  type 'a set
  exception EmptySet

  val empty_set : ('a → 'a → int) → 'a set
  val is_empty : 'a set → bool
  val size : 'a set → int

  val insert : 'a set → 'a → 'a set
  val remove : 'a set → 'a → 'a set

  val union : 'a set → 'a set → 'a set
  val intersect : 'a set → 'a set → 'a set

  val from_list : ('a → 'a → int) → 'a list → 'a set
  val to_list : 'a set → 'a list

  val map : ('a → 'a) → 'a set → 'a set
  val fold : ('a → 'b → 'a) → 'a → 'b set → 'a
end
module FuncSet : FSET
```

(* Bonus *)

```
val ( ++ ) : 'a FuncSet.set → 'a → 'a FuncSet.set
```

- Implementing modules can be tricky as you need to implement *everything* before the module typechecks against its signature. You can make your life easier by either commenting out missing functions from the signature, or by adding empty stubs to the module. For example:

```
module type FSET = sig (* the signature above *) end

module FuncSet: FSET = struct
  type 'a set = ...
  exception EmptySet

  let empty_set _ = failwith "not implemented"
  let is_empty _ = failwith "not implemented"
  let size _ = failwith "not implemented"
  let insert _ _ = failwith "not implemented"
  (* and so on ... *)
end
```

This module will typecheck against FSET despite being completely useless.

- In the official solution, the longest function contains only 10 lines of code.

Problem 2: OToml [25 points]

Let's implement our own pattern-matching engine! We will ~~steal~~ ahem, I mean *borrow* some ideas from OCaml to do that.

For this problem, we will try to implement a subset of TOML that I cooked up one afternoon and named it OToml. Here's an example of a OToml file: basically an INI file) on steroids:

```
[table1]
str = "hello"
int = 5
table = { a = "x", b = "y" }
list = [ 1, 2 ]

[table2]
something = "maybe"
```

The structure of an OToml file is given by the following OCaml definition:

```
type ovalue =
  (* a string constant, e.g., "foo" *)
  | Str of string
  (* an integer constant, e.g., 15 *)
  | Int of int
  (* table, e.g., {name1 = val1, name2 = val2} *)
  | Table of okeyvalue list
  (* list, e.g., [val1, val2] *)
  | ListOf of ovalue list
and okeyvalue = string * ovalue
and otable = string * okeyvalue list
and otoml = otable list
```

Ideally, no key should be repeated in an OToml table, and an OToml file should have no two tables of the same name.

Implement the following functions for manipulating OToml data:

- [3 points] `zip` that takes in two lists, and returns a list of pairs such that its i -th element contains a pair formed from the i -th element of each input list. Return `None` if this cannot be done (e.g., if the list lengths are unequal).
- [3 points] `flatmap` that takes in a function and a list, and returns a list made by concatenating the values obtained by applying the given function on each input list element.
- [4 points] `flatmap_opt` that takes in a function and a list, and returns a list that is a result of concatenating results obtained by applying the given function on each input list element. Note, however, that the input function might return `None` if a transformation fails. In such cases, the output of `flatmap_opt` should also be `None`. See the signature and the sample test case for details.
- [4 points] `get_strings` that returns a list of all string values and their corresponding table names within an OToml record. See the signature and the sample test case for an example.
- [2 points] `is_uppercase` that takes in a string and checks if it is a string formed by uppercase letters. Use `String.uppercase` for this.
- [2 points] `get_uppercase_strings_1` that uses `get_strings` to get all uppercase string values from an OToml record. You must use `get_strings` and the composition operator `%` to implement this function.
 - **Hint:** use the definitions from the sample lecture code to get `%` working.
- [2 points] `get_uppercase_strings_2` that uses `get_strings` to get all uppercase string values from an OToml record. You must use `get_strings` and the pipe operator `>` to implement this function.
- [5 points] Let's define an OToml pattern via the following type:

```
type opattern =
  (* matches anything *)
  | WildcardPattern
  (* matches anything and assigns it to a binding *)
  | BindPattern of string
  (* matches a string *)
```

```

| StringPattern of string
(* matches a integer *)
| IntPattern of int
(* matches a table *)
| TablePattern of (string * opattern) list
(* matches a List [pat1; ...] *)
| ListPattern of opattern list

```

A pattern matches an OToml value if they are of the same shape and type (e.g., `List [Int 1]` matches `ListPattern [IntPattern 1]`, and `String "foo"` matches `StringPattern "foo"`), and if all OToml sub-values match all subpatterns.

Write a function `omatch` that takes in a pattern and an OToml value, and returns a list of matched bindings and their values. If a pattern does not match a value, return `None`. For example, `ListOf [Int 1]` matches `ListPattern [BindPattern "a"]`, and the return value is `Some [("a", Int 1)]`. Furthermore, a `TablePattern` will match a `Table` if both pattern and name of the table items match (unless the name specified in the pattern is an empty string; in that case, the match succeeds only if the patterns match and the name is ignored).

Hint: use `flatmap_opt`.

Bonus

9. [3 points] Write a function `ocheck` that ensures that no two key-value pairs in any OToml table and no two tables in any OToml record share the same name.
10. [7 points] Let's write our own typechecking engine.

```

type otype =
| StrType
| IntType
| ListType of otype
| TableType of otype list

```

Write a function `otypecheck` that, given an OToml record, returns its type or `None` if no type can be inferred. For example, the type of the sample OToml at the beginning of the problem is `TableType [TableType [StrType; IntType; TableType [StrType; StrType]; ListType IntType]; TableType [StrType]]`.

Hints

- The expected type signatures are:

```

val zip : ('a list') → ('b list) → ('a * 'b) list option
val flatmap : ('a → 'b list) → 'a list → 'b list
val flatmap_opt : ('a → 'b list option) → 'a list → 'b list option
val get_strings : otoml → (string * string) list
val is_uppercase : string → bool
val get_uppercase_strings_1 : otoml → string list
val get_uppercase_strings_2 : otoml → string list
val omatch : opattern → ovalue → (string * ovalue) list option

(* Bonus *)
val ocheck : otoml → bool
val otypecheck : otoml → otype list option

```

- In the official solution, the longest function contains fewer than 20 lines of code. Your mileage might vary, though.

Testing

Suppose that you want to test the first problem (`prob1.ml`). You should create a file `test_prob1.ml` with the following contents:

```
open Prob1 (* this is the _only_ place you should use `open` in this assignment *)

let test_is_empty () =
  let cmp = Int.compare in
  let set = FuncSet.empty_set cmp in
  assert (FuncSet.is_empty set = true);
  let set = FuncSet.insert set 1 in
  assert (FuncSet.is_empty set = false);
let _ = test_is_empty ()

(* you should create similar functions with many test cases for other tasks as well *)
```

Once done, you can create an executable with the following command:

```
ocamlc -o prob1 prob1.ml test_prob1.ml # This compiles source files into an executable
./prob1 # This line executes the compiled executable
```

If any of your tests fail, you will get the following error:

```
Fatal error: exception Assert_failure("test_prob1.ml", 5, 2)
```

indicating that you have a problem at line 5 of `test_prob1.ml`.

You can also use `utop` if you prefer to test stuff manually. Another idea is to use a proper unit testing framework such as `OUnit` ([details are available here](#)). Note that the latter choice is recommended only for adventurous souls— it goes way beyond the stuff we’ve learned thus far.

Sample test cases

```
(* Problem 1 *)

let cmp = Int.compare (* use ascending integer comparator for simplicity *)
let set = FuncSet.empty_set cmp

FuncSet.is_empty set ;;
(* true *)
FuncSet.size set ;;
(* 0 *)
FuncSet.to_list set ;;
(* [] *)

let set = FuncSet.insert set 1 ;;

FuncSet.size set ;;
(* 1 *)

FuncSet.to_list set ;;
(* [1] *)
FuncSet.to_list (FuncSet.insert set (-1)) ;;
(* [-1; 1] *)
FuncSet.to_list (FuncSet.insert set 1) ;;
(* [1] *)

(* FuncSet.to_list (...) gets annoying. You can instead use application operator @@.
For example, f (expr) is equivalent to f @@ expr *)

let set2 = FuncSet.insert (FuncSet.insert (FuncSet.insert (FuncSet.empty_set cmp) 1) 2) (-1)

FuncSet.to_list @@ FuncSet.union set set2;
(* [-1; 1; 2] *)
```

```

FuncSet.to_list @@ FuncSet.intersect set set2;
(* [1] *)

FuncSet.to_list @@ FuncSet.from_list cmp [100; 1; 2; -2; 2; 1];
(* [-2; 1; 2; 100] *)

FuncSet.to_list @@ FuncSet.map (fun x → x mod 2) set2
(* [-1; 0; 1] *)

FuncSet.fold (fun acc x → acc + x) 0 set2
(* 2 *)

let open FuncSet in (* Local open statement to avoid typing FuncSet all the time below *)
to_list @@ empty_set cmp ++ 1 ++ 2 ++ 3 ++ 1
(* [1; 2; 3] *)

(* Problem 2 *)

zip [1; 2; 3] ["a"; "b"; "c"]
(* Some [(1, "a"); (2, "b"); (3, "c")] *)

zip [1; 2] ["a"]
(* None *)

flatMap (fun x → [x; -x]) [1; 2; 3]
(* [1; -1; 2; -2; 3; -3] *)

flatMap_opt (fun x → if x mod 2 = 0 then Some [x; -x] else None) [2; 4]
(* Some [2; -2; 4; -4] *)

flatMap_opt (fun x → Some x) []
(* Some [] *)

flatMap_opt (fun x → if x mod 2 = 0 then Some [x; -x] else None) [1; 2; 4]
(* None *)

(* A value corresponding to the sample file in Problem 2 *)
let t = [ "table1", [ ("str", Str "hello")
                    ; ("int", Int 5)
                    ; ("table", Table ["a", Str "x"; "b", Str "y"])
                    ; ("list", ListOf [Int 1; Int 2]) ]
        ; "table2", [ ("something", Str "maybe" ) ] ]

get_strings t
(* [("table1", "hello"); ("table1", "x"); ("table1", "y"); ("table2", "maybe")] *)

is_uppercase "HELLO"
(* true *)
is_uppercase "Hello"
(* false *)

get_uppercase_strings_1 t
(* ["y"] *)

let p = TablePattern
[ "", StringPattern "hello"
; "int", BindPattern "x"
; "", TablePattern [ "", BindPattern "y"; "b", WildcardPattern ]
; "", WildcardPattern ]

```

```

omatch p (Table (snd @@ List.hd t))
(* Some [("x", Int 5); ("y", Str "x")] *)

let p = TablePattern
  [ "", StringPattern "hello"
  ; "", BindPattern "x"
  ; "", TablePattern [ "a", BindPattern "y"; "c", WildcardPattern ] ]
omatch p (Table (snd @@ List.hd t))
(* None *)

(* Bonus *)

ocheck t
(* true *)

let t' = (List.hd t) :: ["table1", []] in
ocheck t'
(* false *)

otypecheck t
(* Some [ TableType [ StrType; IntType; TableType [ StrType; StrType ]; ListType IntType ]
; TableType [ StrType ] ] *)
otypecheck (t @ ["table3", ["foo", ListOf [Int 1; Str "2"]]])
(* None *)

```